I would make one important change before freezing the schema.

Don’t store everything as separate tables if it represents the same timeline.
Instead, organize the database into **five logical layers**:

1. Identity & Academic Data
2. Session Management
3. High-Frequency Time-Series (TimescaleDB)
4. Semantic Events & State Changes
5. AI/Agent Analytics

This is closer to how production telemetry systems are designed.

Final FocusAI Database Schema v1.0

Database

PostgreSQL 17

Extensions

- TimescaleDB
- pgcrypto
- uuid-ossp
- JSONB

Layer 1 — Identity & Academic Data (PostgreSQL)

students

student_id UUID PRIMARY KEY

roll_number

first_name

last_name

email

department

semester

created_at

updated_at

---

instructors

instructor_id UUID PRIMARY KEY

name

email

department

created_at

---

courses

course_id UUID PRIMARY KEY

course_code

```
course_name

department

semester

instructor_id FK
```

---

users

```
user_id UUID PRIMARY KEY

username

password_hash

role

created_at
```

Roles

```
ADMIN

FACULTY

STUDENT
```

Layer 2 — Session Data

sessions

session_id UUID PRIMARY KEY

student_id FK

course_id FK

device

browser

os

camera_resolution

fps

start_time

end_time

duration

created_at

Layer 3 — High Frequency Telemetry (TimescaleDB Hypertables)

Partition Column

time TIMESTAMPTZ

---

attention_metrics

time

session_id

attention_score

attention_state

confidence

attention_state

Focused

Distracted

Fatigued

---

emotion_metrics

Stores raw model predictions.

time

session_id

emotion

confidence

emotion

Interested

Confused

Bored

gaze_metrics

time

session_id

gaze_x

gaze_y

fixation_duration

saccade_velocity

eye_closed

facial_metrics

time

session_id

EAR

MAR

blink_rate

smile_score

smile_type

eyebrow_raise

eyebrow_lower

eyebrow_asymmetry

AU6

AU12

AU14

lip_corner_angle

lip_compression

mouth_asymmetry

drooping_lip_score

head_pitch

head_roll

head_yaw

---

body_metrics

time

session_id

neck_angle

shoulder_angle

body_pitch

body_roll

left_hand

right_hand

posture

movement_score

Layer 4 — Semantic Events (TimescaleDB)

These tables store **meaningful behavioral events**, not raw frame data.

---

behavioral_events

event_id UUID

time

session_id

event_type

start_time

end_time

duration_ms

confidence

metadata JSONB

Supported Event Types

NO_FACE

MULTIPLE_FACE

FACE_OCCLUDED

LOOK_LEFT

LOOK_RIGHT

LOOK_UP

LOOK_DOWN

HEAD_TURN

LONG_EYE_CLOSURE

BLINK_BURST

YAWN

GENUINE_SMILE

FAKE_SMILE

CONTEMPT

DROOPING_LIPS

HAND_RAISED

POSTURE_CHANGE

LIVENESS_FAILED

SPOOF_ATTEMPT

---

emotion_segments (NEW)

Stores continuous emotion periods.

segment_id UUID

session_id

emotion

start_time

end_time

duration_ms

avg_confidence

trigger_reason JSONB

Example

Emotion	Start	End
Interested	10:00	10:08
Confused	10:08	10:11
Interested	10:11	10:25
Bored	10:25	10:31

---

emotion_transitions

transition_id UUID

session_id

time

previous_emotion

current_emotion

confidence

reason JSONB

Example

Interested

Confused

Interested

Bored

---

attention_segments

segment_id UUID

session_id

attention_state

start_time

end_time

duration_ms

average_score

Example

Focused

Distracted

Focused

Fatigued

---

liveness_checks

time

session_id

challenge

expected_action

detected_action

success

confidence

response_time_ms

---

warnings

warning_id UUID

time

session_id

warning_type

message

severity

resolved

Example

Face Missing

Multiple Faces

Spoof Attempt

Camera Blocked

Low Attention

Layer 5 — AI & Agent Analytics

agent_analysis

analysis_id UUID

session_id

time

attention_reason

emotion_reason

engagement_reason

confidence

explanation JSONB

Example JSON

```
{
  "attention_drop": [
    "Looking away",
    "Head down",
    "Blink burst"
  ],
  "emotion": "Confused",
  "confidence": 0.91
}
```

---

engagement_summary

One row per session.

session_id

average_attention

highest_attention

lowest_attention

average_interest

average_confusion

average_boredom

total_yawns

total_blinks

looking_away_count

emotion_changes

attention_changes

engagement_score

---

recommendations

recommendation_id UUID

session_id

generated_at

priority

recommendation

status

Example

Review topic after minute 21

Student fatigued

Increase interaction

---

reports

report_id UUID

session_id

generated_at

report_type

summary JSONB

pdf_path

Complete Database Flow

Students

Sessions

Raw Telemetry (TimescaleDB)

Attention

Emotion

Gaze

Face

Body

Event Engine

Behavioral Events

Emotion Segments

Attention Segments

Emotion Transitions

Warnings

Agent Layer

Analytics

Recommendations

Reports

Dashboard

Final Table Count

PostgreSQL Tables (7)

1. students
2. instructors
3. courses
4. users
5. sessions
6. recommendations
7. reports

TimescaleDB Hypertables (10)

8. attention_metrics
9. emotion_metrics
10. gaze_metrics
11. facial_metrics
12. body_metrics
13. behavioral_events
14. emotion_segments
15. emotion_transitions
16. attention_segments
17. liveness_checks
18. warnings

Why this is the final design

This schema captures three different levels of information:

- **Raw telemetry** for research, visualization, and model improvement.
- **Behavioral state changes** with exact start times, end times, durations, and transitions (e.g., when boredom started, when it ended, emotion changes, and yawning intervals).

- **High-level AI reasoning** from your agent layer for explanations, recommendations, and reports.

It gives you enough detail to answer questions like:

- *When did the student first become bored?*
- *How long did each boredom episode last?*
- *Did yawning occur before or after boredom?*
- *How many emotion transitions happened during the lecture?*
- *What events contributed to attention drops?*

while remaining efficient enough for real-time analytics on TimescaleDB.